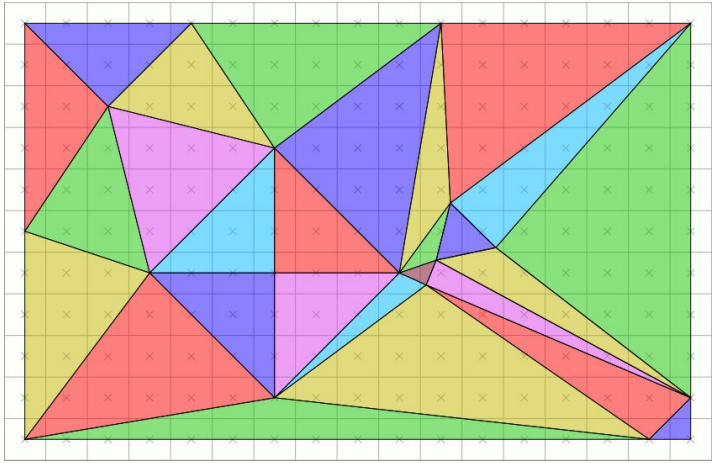
CSL 7450
Computer Graphics

Fundamentals

Goal in Graphics

- Visualizing a virtual world on a 2D screen
- The screen: Essentially a grid of pixels
- We need to define a 3D array of dimensions (Width x Height x 3)
- Can be done manually on CPU and then visualized using any library such as opencv
- Where does OpenGL come in?



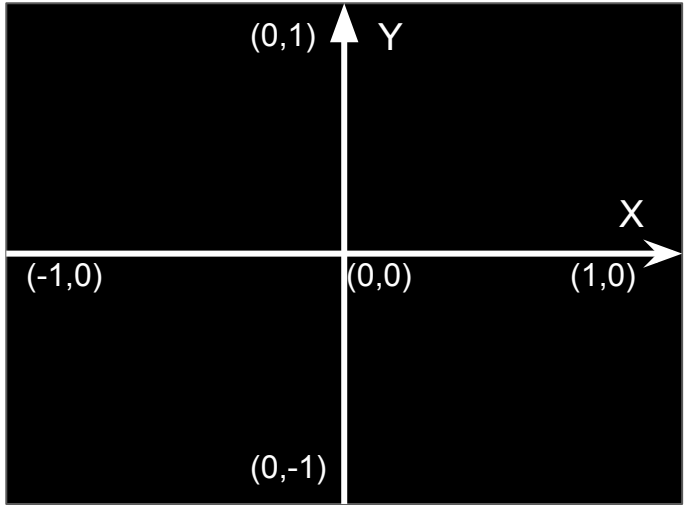
Role of OpenGL

- Deciding the colour of each pixel iteratively using CPU takes time

- GPU based pipeline

- Our role:
  - Define Geometry as a set of primitives (point, line or triangle) by defining the vertices and edges
  - Write a program to process one primitive and convert it to a 2D point

- OpenGL coordinate system: **[-1,1]** in each dimension



Modern OpenGL components

1. **Vertex Buffer Object (VBO):**
    a. A chunk of memory that stores all the vertices of an object on the GPU
2. **Index/Element Buffer Object (EBO / IBO):**
    a. To store indices as to which vertices of VBO form primitives (ie- edge data)
3. **Vertex Array Object (VAO):**
    a. To store the attributes layout of a VBO (how many floats per vertex, etc.)
4. **Shaders (and uniforms):**
    a. A program that runs on the GPU
    b. 2 Shaders to write (Vertex Shader and Fragment Shader)
    c. Language: glsl (OpenGL shader language)

<u>Will need to call them very often so we'll create wrapper class around each of them for abstraction</u>

OpenGL in Python

Libraries Required:

1. **PyOpenGL** (for opengl function calls)
2. **PyOpenGL_accelerate** (for speeding up pyopengl operations)
3. **glfw** (to create window. Alternatives: pygame, pyqt5)
4. **numpy** (for vector and matrices)
5. **pyrr** (Optional: To easily create common transformation matrices for us)

Setup:

```
pip install PyOpenGL PyOpenGL_accelerate pyrr glfw
```

Demo_0: glfw window

1. Initialize glfw

```
glfw.init()
glfw.window_hint(glfw.CONTEXT_VERSION_MAJOR, 3)
glfw.window_hint(glfw.CONTEXT_VERSION_MINOR, 3)
glfw.window_hint(glfw.OPENGL_PROFILE, glfw.OPENGL_CORE_PROFILE)
glfw.window_hint(glfw.OPENGL_FORWARD_COMPAT, GL_TRUE)
```

2. Create window

```
height = 600
width = 800
window = glfw.create_window(
    width, height,
    "Demo", None, None)
```

3. Activate window

```
glfw.set_window_pos(window, 500, 200)
glfw.make_context_current(window)
glViewport(0, 0, width, height)
```

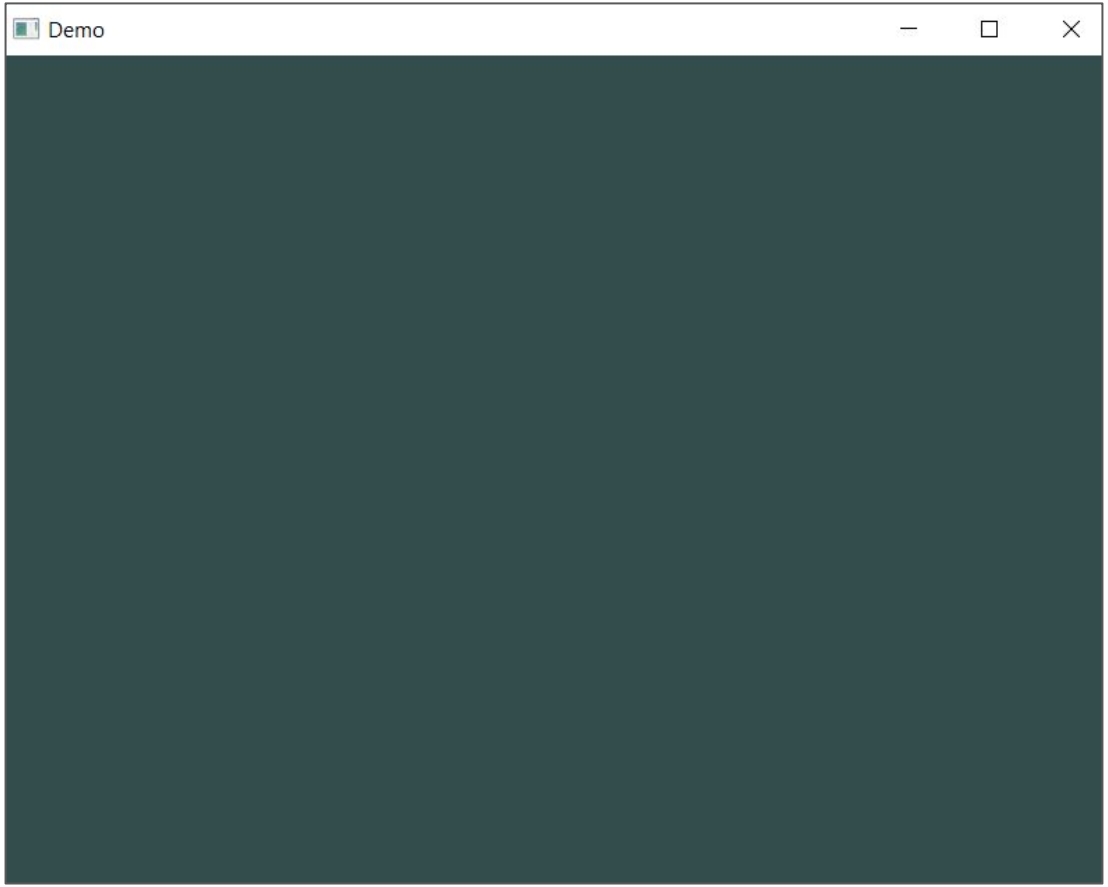
4. Main Render Loop

```
while not glfw.window_should_close(window):
    glfw.poll_events()


    glClearColor(0.2, 0.3, 0.3, 1.0) # RGBA
    glClear(GL_COLOR_BUFFER_BIT)

    # ----------
    # Render here
    # ----------

    glfw.swap_buffers(window)
```

5. Terminate glfw

```
glfw.terminate()
```

Demo_0: glfw window



VBO Wrapper Class

OpenGL provides an unsigned int ID for every object created on the GPU

```
class VBO:
    def __init__(self, vertices):
        self.ID = glGenBuffers(1)
        glBindBuffer(GL_ARRAY_BUFFER, self.ID)
        glBufferData(GL_ARRAY_BUFFER,
                     vertices.nbytes, vertices,
                     GL_STATIC_DRAW)
    def Use(self):
        glBindBuffer(GL_ARRAY_BUFFER, self.ID)
    def Delete(self):
        glDeleteBuffers(1, (self.ID,))
```

```
vertices = np.array([
    -0.5, -0.5, 0.0,
    0.5, -0.5, 0.0,
    0.0, 0.5, 0.0
], dtype = np.float32)
```

Example: Vertices of a triangle

IBO Wrapper Class

OpenGL provides an unsigned int ID for every object created on the GPU

```
class IBO:
    def __init__(self, indices):
        self.ID = glGenBuffers(1)
        self.count = len(indices)
        glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, self.ID)
        glBufferData(GL_ELEMENT_ARRAY_BUFFER,
                     indices.nbytes, indices,
                     GL_STATIC_DRAW)
    def Use(self):
        glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, self.ID)
    def Delete(self):
        glDeleteBuffers(1, (self.ID,))
```

```
indices = np.array([
    0,1,2
], dtype = np.uint32)
```

Example: Indices of a triangle

VAO Wrapper Class

VAO needs a VBO to describe. Once described, binding the VAO automatically binds the corresponding VBO.

```
class VAO:
    def __init__(self, vbo : VBO):
        self.vao = glGenVertexArrays(1)
        glBindVertexArray(self.vao)
        vbo.Use()
        glEnableVertexAttribArray(0)
        glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE,
        ctypes.c_uint(3 * ctypes.sizeof(ctypes.c_float)),
            ctypes.c_void_p(0))
    def Use(self):
        glBindVertexArray(self.vao)
    def Delete(self):
        glDeleteVertexArrays(1, (self.vao,))
```

Shader Wrapper Class

2 Types of shaders: Vertex and Fragment.
Need to be written as a string, and then compiled into a shader program

```
class Shader:
    def __init__(self, vertex_shader, fragment_shader):
        self.ID = compileProgram(
            compileShader(vertex_shader, GL_VERTEX_SHADER),
            compileShader(fragment_shader, GL_FRAGMENT_SHADER)
            )
        self.Use()
    def Use(self):
        glUseProgram(self.ID)
    def Delete(self):
        glDeleteProgram((self.ID,))
```

Vertex Shader

```
#version 330 core
layout(location = 0) in vec3 vertexPosition;
void main() {
    gl_Position = vec4(vertexPosition, 1.0);
}
```

- On issuing draw call: Vertices from the VBO get processed in parallel (Runs once for each vertex).
- **Input:** The layout of each vertex as defined in VAO is available as input to Vertex Shader
- **Special variable "gl_Position"** to be set as normalized coordinates of that vertex (between -1 to +1 along each dimension, automatically mapped to output window height and width).
- **Output:** If explicitly set, can output variables to the fragment shader.

Fragment Shader

```
#version 330 core
out vec4 FragColor;
void main() {
    FragColor = vec4(1.0, 1.0, 1.0, 1.0); // Set color
}
```

- Runs once for **each pixel**.
- **Input: output from Vertex Shader** gets interpolated for this pixel and is available here as input
- **Output**: Define a vec4 output variable and set it to be the output colour (RGBA) of the pixel.

Structure of OpenGL Programs

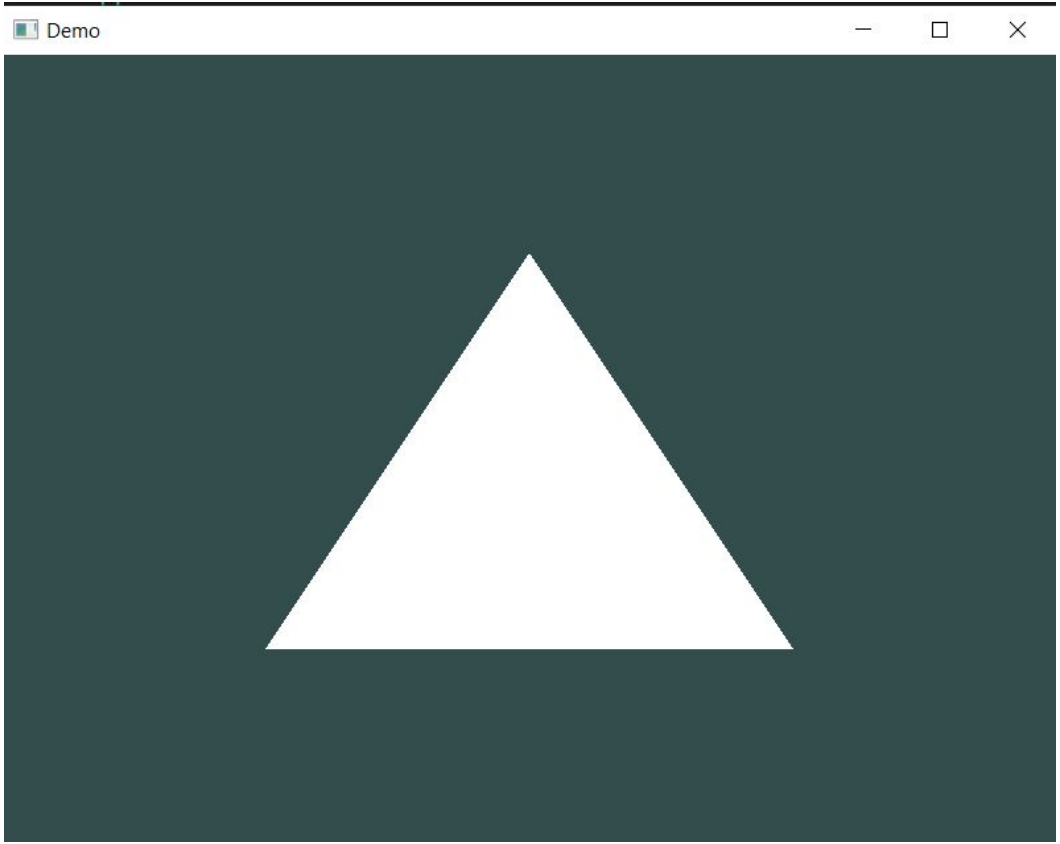
1. Initialize OpenGL context and create a window
2. Set up Geometry data (VBO, EBO, VAO)
3. Set up Shaders

4. Render loop:
   a. Clear the screen
   b. Bind shaders and buffers
   c. Draw primitives (e.g., triangles)
5. Clean up resources

```
vbo = VBO(vertices)
ibo = IBO(indices)
vao = VAO(vbo)
shader = Shader(vertex_shader,
                fragment_shader)
```

```
# ----------

# Bind the shader, vao (automatically binds vbo) and ibo
shader.Use()
vao.Use()
ibo.Use()

# Issue Draw call with primitive type
glDrawElements(GL_TRIANGLES, ibo.count, GL_UNSIGNED_INT, None)

# ----------
```

Demo_1: Simple Triangle



Uniforms in Shaders

```
uniform mat4 modelMatrix;
```

```
gl_Position = modelMatrix * vec4(vertexPosition, 1.0);
```

- Variables common for all vertices being processed in a draw call
- Set from the CPU side before the draw call (After the shader is binded)

```
shader.Use()
modelMatrixLocation = glGetUniformLocation(shader.ID, "modelMatrix".encode('utf-8'))
glUniformMatrix4fv(modelMatrixLocation, 1, GL_TRUE, model_matrix)
```

Demo_2: Moving Coloured Square

Square geometry (with colour)

```python
# Define Geometry
vertices = np.array([
    -0.5, -0.5, 0.0, 1.0, 0.0, 0.0,
    0.5, -0.5, 0.0, 0.0, 1.0, 0.0,
    0.5, 0.5, 0.0, 0.0, 0.0, 1.0,
    -0.5, 0.5, 0.0, 1.0, 0.0, 1.0
], dtype = np.float32)

indices = np.array([
    0,1,2,
    0,3,2
], dtype = np.uint32)
```

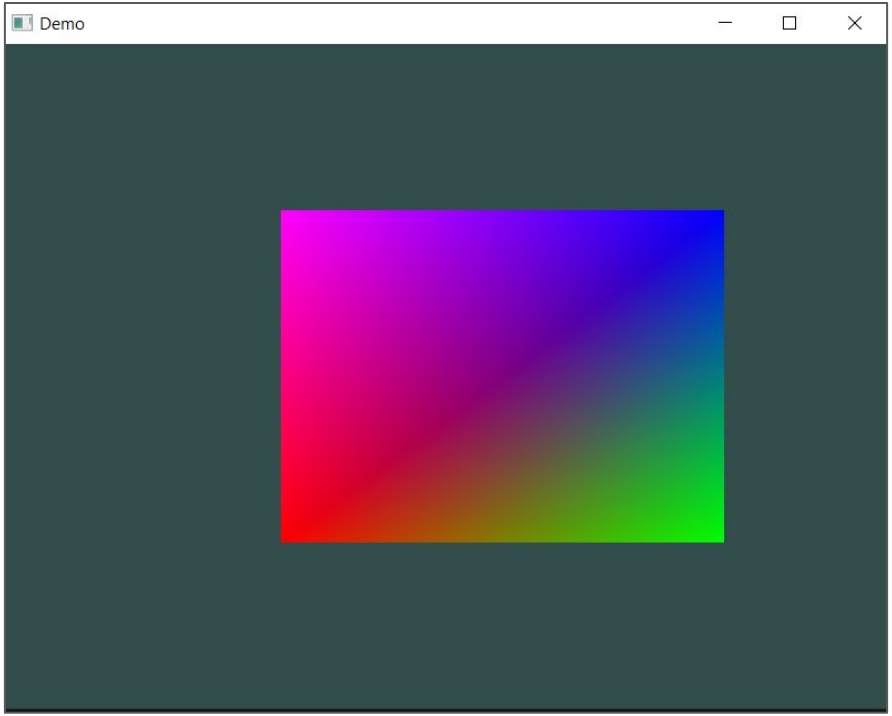
```
#version 330 core
layout(location = 0) in vec3 vertexPosition;
layout(location = 1) in vec3 vertexColour;
out vec3 fragmentColour;
uniform mat4 modelMatrix;
void main() {
    fragmentColour = vertexColour;
    gl_Position = modelMatrix * vec4(vertexPosition, 1.0);
}
```

Vertex Shader

```
#version 330 core
in vec3 fragmentColour;
out vec4 outputColour;
void main() {
    outputColour = vec4(fragmentColour, 1.0); // Set color
}
```

Fragment Shader

Demo_2: Moving Coloured Rectangle???



- Why is it a rectangle and not a square?

Drawing Multiple Objects (Scene)

- Need a VAO, VBO, IBO, Shader for each object.
- Might get tedious to write.
- Abstract out further

```python
class Object:
    def __init__(self, shader, properties):
        self.properties = copy.deepcopy(properties)

        self.vbo = VBO(self.properties['vertices'])
        self.ibo = IBO(self.properties['indices'])
        self.vao = VAO(self.vbo)

        self.properties.pop('vertices')
        self.properties.pop('indices')

        # Create shaders
        self.shader = shader
```

```python
obj1Props = {
    'vertices' : np.array(obj1Verts, dtype = np.float32),

    'indices' : np.array(obj1Inds, dtype = np.uint32),

    'position' : np.array([0, 0, 0], dtype = np.float32),

    'rotation_z' : 0.0,

    'scale' : np.array([30, 30, 1], dtype = np.float32),

    'sens' : 125,

    'velocity' : np.array([0, 0, 0], dtype = np.float32)
}
```

Drawing Multiple Objects (Contd.)

```
# Bind the shader, set uniforms, bind vao (automatically binds vbo) and ibo
self.shader.Use()
modelMatrixLocation = glGetUniformLocation(self.shader.ID, "modelMatrix".encode('utf-8'))
glUniformMatrix4fv(modelMatrixLocation, 1, GL_TRUE, model_matrix)

self.vao.Use()
self.ibo.Use()

# Issue Draw call with primitive type
glDrawElements(GL_TRIANGLES, self.ibo.count, GL_UNSIGNED_INT, None)
```

Scene Camera

- Can abstract out setting Camera Matrix in each shader as a Camera Class

```
class Camera:
    def __init__(self, height, width):
        self.height = height
        self.width = width
    def Update(self, shader):
        shader.Use()

        camMatrix = np.array([[2.0/self.width, 0,0,0],
                              [0,2.0/self.height,0,0],
                              [0,0,1,0],
                              [0,0,0,1]], dtype = np.float32)

        camMatrixLocation = glGetUniformLocation(shader.ID, "camMatrix".encode('utf-8'))
        glUniformMatrix4fv(camMatrixLocation, 1, GL_TRUE, camMatrix)
```

Scene Elements

Elements of our Scene

1. Shaders
2. Camera
3. Objects

```
self.camera = Camera(self.windowHeight, self.windowWidth)
self.shaders = [Shader(objShaderSource["vertex_shader"],
                       objShaderSource["fragment_shader"])]
self.objects = [Object(self.shaders[0], obj1Props)]
```

Drawing

1. Update Camera
2. Draw all objects

```
# Update the camera matrix in shaders
for shader in self.shaders:
    self.camera.Update(shader)

# Draw the object
for obj in self.objects:
    obj.Draw()
```